

Student Management System Using MongoDB

A NoSQL mini-project demonstrating core MongoDB operations (CRUD, comparison and logical operators, sorting, limiting and counting) on a collection of student records.

University	Babu Banarasi Das University (2026-2027)
School	School of Computer Applications
Department	BCA (DS & AI)
Submitted by	Gulshan Singh
Class Roll No	42
University Roll No	1250258207
Section	BCADS23
Submitted to	Mr. Harendra Singh

Overview

MongoDB is a document-oriented NoSQL database that stores data as flexible JSON-like documents instead of fixed rows and columns. In this project, a Student Management System is built on MongoDB with 30 student records containing roll number, name, age, marks and city. Different queries are then run to insert, retrieve, filter, update, sort, count and delete data.

Objectives

- Understand the concept of NoSQL databases
- Learn the basic features of MongoDB
- Create a database and a collection
- Insert 30 student records
- Perform CRUD operations
- Use comparison operators (\$eq, \$gt, \$lt, \$gte, \$lte)
- Perform sorting and filtering
- Understand logical AND/OR operations

Tech Stack

- **Database:** MongoDB
- **Shell:** mongosh (MongoDB Shell)

Database Design

Database: Students | **Collection:** students

Field	Type	Description
_id	ObjectId	Auto-generated unique ID
roll	Number	Student roll number (101-130)
name	String	Student name
age	Number	Age (18-20)
marks	Number	Marks obtained
city	String	City of residence

Sample document:

```
{
  roll: 101,
  name: "Shrishti Singh",
  age: 19,
  marks: 85,
  city: "Delhi"
}
```

Getting Started

Prerequisites:

- MongoDB Community Server
- MongoDB Shell (mongosh)

Run the project:

```
# 1. Start the MongoDB shell
mongosh

# 2. Run the queries from this repository
# (paste them one by one, or load a script file)
load("queries.js")
```

Operations Covered

#	Operation	Description
1	Create Database	use Students
2	Create Collection	db.createCollection("students")
3	Insert 30 Students	insertMany()
4	Display All Students	find()

#	Operation	Description
5	Equal To	\$eq
6	Greater Than	\$gt
7	Less Than	\$lt
8	Greater Than or Equal To	\$gte
9	Less Than or Equal To	\$lte
10	Update	updateOne()
11	Delete One	deleteOne()
12	Delete Many	deleteMany()
13	AND Operation	\$and
14	OR Operation	\$or
15	Sort	sort() ascending and descending
16	Limit	limit()
17	Search by City	find({city:...})
18	Count Documents	countDocuments()

Query Reference

```
// 1. Create / switch database
use Students

// 2. Create collection
db.createCollection("students")

// 3. Insert students (30 records in total - first few shown)
db.students.insertMany([
  { roll: 101, name: "Shrishti Singh", age: 19, marks: 85, city: "Delhi" },
  { roll: 102, name: "Prathana Bajaj", age: 18, marks: 72, city: "Lucknow" },
  { roll: 103, name: "Rhimsha Khan", age: 18, marks: 62, city: "Ghaziabad" }
  // ... up to roll 130
])

// 4. Display all students
db.students.find()

// 5-9. Comparison operators
db.students.find({ marks: { $eq: 88 } })
db.students.find({ marks: { $gt: 85 } })
db.students.find({ marks: { $lt: 60 } })
db.students.find({ marks: { $gte: 90 } })
db.students.find({ marks: { $lte: 70 } })
```

```
// 10. Update
db.students.updateOne({ roll: 105 }, { $set: { marks: 95 } })

// 11. Delete one
db.students.deleteOne({ roll: 110 })

// 12. Delete many (marks below 60)
db.students.deleteMany({ marks: { $lt: 60 } })

// 13. AND
db.students.find({ $and: [ { marks: { $gt: 80 } }, { age: 19 } ] })

// 14. OR
db.students.find({ $or: [ { city: "Lucknow" }, { city: "Delhi" } ] })

// 15. Sort
db.students.find().sort({ marks: 1 }) // ascending
db.students.find().sort({ marks: -1 }) // descending

// 16. Limit
db.students.find().limit(5)

// 17. Search by city
db.students.find({ city: "Lucknow" })

// 18. Count documents
db.students.countDocuments()
```

Note: After the update and delete operations, the collection holds 27 documents (roll 110 deleted individually; rolls 111 and 113 removed by the marks < 60 filter).

Operators Used

Operator	Meaning	Example
\$eq	Equal to	marks: { \$eq: 88 }
\$gt	Greater than	marks: { \$gt: 85 }
\$lt	Less than	marks: { \$lt: 60 }
\$gte	Greater than or equal to	marks: { \$gte: 90 }
\$lte	Less than or equal to	marks: { \$lte: 70 }
\$and	All conditions true	\$and: [...]
\$or	At least one condition true	\$or: [...]

Repository Structure

```
README.md
queries.js           # All MongoDB queries (optional script)
No_sql_Project_Report.pdf # Full project report with screenshots
No_sql_Second_Assignment.pdf # Assignment submission
```

Conclusion

This project gives hands-on practice with MongoDB: creating a database and collection, inserting documents, querying with comparison and logical operators, updating and deleting records, and sorting, limiting and counting results. It shows how MongoDB can manage and query student data efficiently.

Submitted by **Gulshan Singh**, BCA (DS & AI), Babu Banarasi Das University.